

U.E 5.1 - Intelligence Artificielle : Machine learning

Promotion FISE 2027

ENSTA BRETAGNE
2 Rue François Verny
29200 Brest Cedex 9, France



BARROS ALVES Luiz Felipe
luiz.barros@ensta.fr

Sommaire

1	Linear regression with one variable	2
1.1	Plotting the Data	2
2	Gradient Descent	2
2.1	Computing the cost $J(\theta)$	2
2.2	Gradient descent	3
2.3	Visualizing $J(\theta)$	5
3	Linear regression with multiple variables	5
3.1	Feature Normalization	5
3.2	Learning with gradient descent	6
3.2.1	Compute cost and gradients	6
3.2.2	Selecting learning rates	8
3.2.3	Predict the price for a new house	8
3.3	Optional exercise: Normal Equations	9

1 Linear regression with one variable

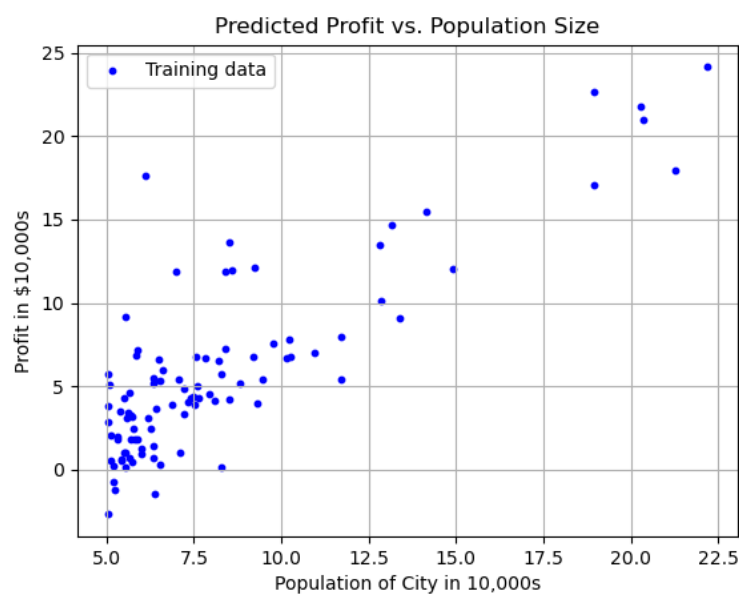
1.1 Plotting the Data

- How many features are involved in this problem?
- How much data is involved in this problem?
- What is the problem from a machine learning point-of-view?

Supervised/Unsupervised?

Regression/Classification?

Linear/?on-linear problem



2 Gradient Descent

2.1 Computing the cost $J(\theta)$

```
1 import numpy as np
2
3 def computeCost(X, y, theta):
4     """
5         computes the cost of using theta as the parameter for
6         linear
7         regression to fit the data points in X and y
8     """
9     m = y.size
10    J = 0.
```

```

11 # =====YOUR CODE HERE =====
12 # Instructions: Compute the cost of a particular choice of
    theta
13
14
15 #h = float(theta[0,0]) + float(theta[1,0])*X[:,1]
16 #h = h.reshape((m,1))
17 h = X @ theta
18 J = (1/(2*m)) * np.sum((h-y)**2)
19
20 # =====
21
22 return J

```

2.2 Gradient descent

```

1 import numpy as np
2 from computeCost import computeCost
3
4 def gradientDescent(X, y, theta, alpha, num_iters):
5     """
6     Performs gradient descent to learn theta
7     theta, cost_history, theta_history = gradientDescent(X,
8     y, theta, alpha, num_iters) updates theta by
9     taking num_iters gradient steps with learning rate
10    alpha
11    """
12    # Initialize some useful values
13    m = y.size # number of training examples
14    n = theta.size # number of parameters
15    cost_history = np.zeros(num_iters) # cost over iters
16    theta_history = np.zeros((n,num_iters)) # theta over iters
17
18    for i in range(num_iters):
19        # ===== YOUR CODE HERE =====
20
21        # Instructions: Perform a single gradient step on the
22        parameter vector
23        theta.
24
25        # Hint: While debugging, it can be useful to print out the
26        values
27        of the cost function (computeCost) and gradient
28        here.
29
30        h = X @ theta

```

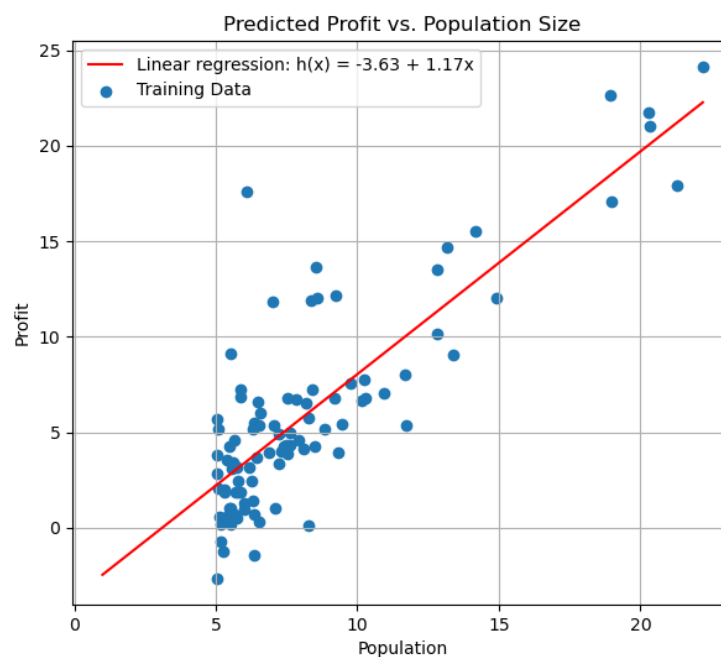
```

26     theta[0,0] = theta[0,0] - (alpha/m)*np.sum( ((h-y)*X
27           [:,0].reshape((m,1))) )
28     theta[1,0] = theta[1,0] - (alpha/m)*np.sum(((h-y)*X
29          [:,1].reshape((m,1))))
30
31     cost_history[i] = computeCost(X, y, theta)
32     theta_history[:,i] = theta.reshape((2,))
33
34     #
35     =====
36
37     return theta, cost_history, theta_history

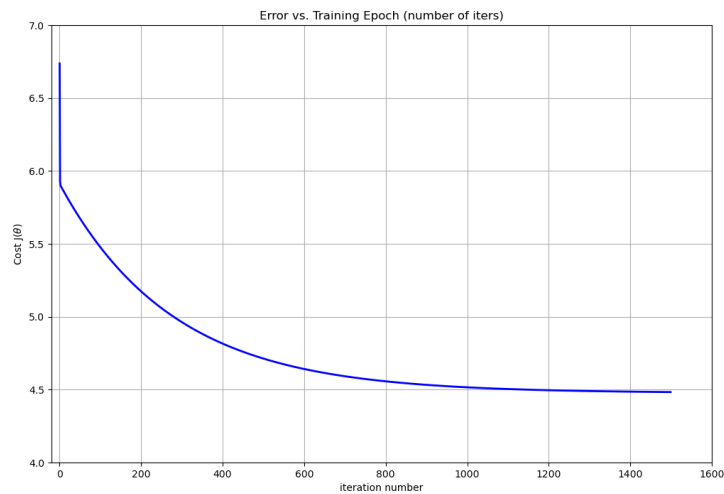
```

As you program, make sure you understand what you are trying to optimize and what is being updated.

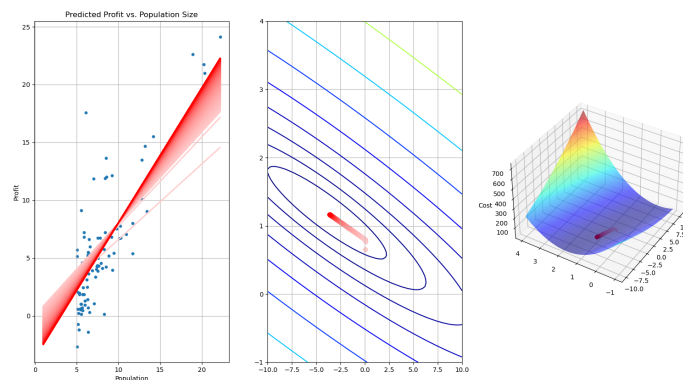
Training data with linear regression fit :



2.3 Visualizing $J(\theta)$



, but you should understand how the code you have written already is creating these images. In the next step of ex1.py



3 Linear regression with multiple variables

3.1 Feature Normalization

```

1 import numpy as np
2
3
4 def featureNormalize(X):
5     """
6         returns a normalized version of X where
7         the mean value of each feature is 0 and the standard
            deviation
    """

```

```

8         is 1. This is often a good preprocessing step to do
9         when
10        working with learning algorithms.
11    """
12    X_norm, mu, sigma = 0,0,0
13    # ===== YOUR CODE HERE
14    # =====
15    # Instructions: First, for each feature dimension, compute
16    # the mean
17    # of the feature and subtract it from the
18    # dataset,
19    # storing the mean value in mu. Next,
20    # compute the
21    # standard deviation of each feature and
22    # divide
23    # each feature by it's standard deviation,
24    # storing
25    # the standard deviation in sigma.
26    #
27    # Note that X is a matrix where each column
28    # is a
29    # feature and each row is an example. You
30    # need
31    # to perform the normalization separately
32    # for
33    # each feature.
34    #
35    # Hint: You might find the 'mean' and 'std' functions
36    # useful.
37    #
38
39    mu = np.mean(X, axis=0)
40    sigma = np.std(X, axis=0)
41    X_norm = (X - mu) / sigma
42
43    # =====
44
45    return X_norm, mu, sigma

```

3.2 Learning with gradient descent

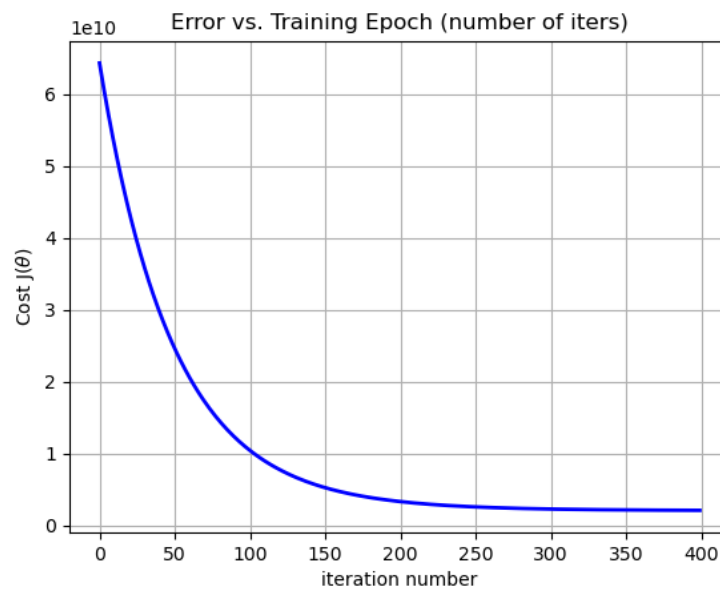
3.2.1 Compute cost and gradients

```

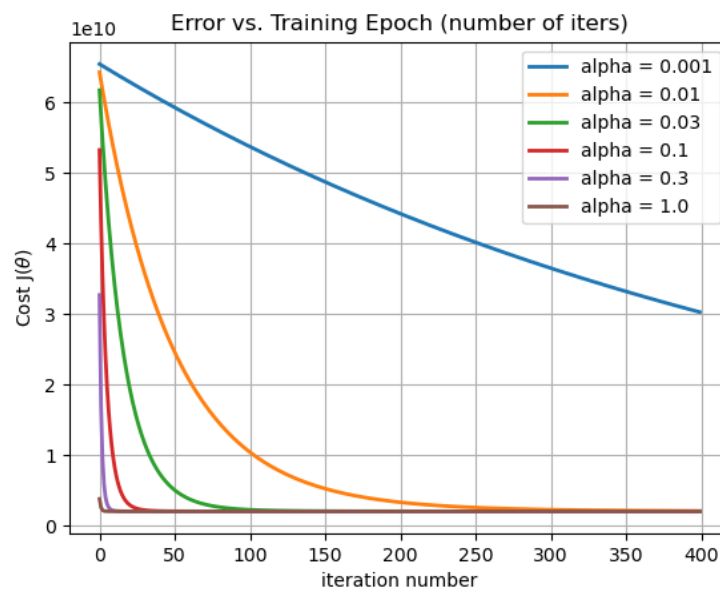
1 import numpy as np
2 from computeCostMulti import computeCostMulti
3

```

```
4 def gradientDescentMulti(X, y, theta, alpha, num_iters):
5     """
6     Performs gradient descent to learn theta
7     theta = gradientDescent(x, y, theta, alpha, num_iters)
8     updates theta by
9     taking num_iters gradient steps with learning rate
10    alpha
11    """
12    # Initialize some useful values
13    m = y.size # number of training examples
14    n = theta.size # number of parameters
15    cost_history = np.zeros(num_iters)
16    theta_history = np.zeros((n,num_iters))
17
18    for i in range(num_iters):
19        # ===== YOUR CODE HERE =====
20        # Instructions: Perform a single gradient step on the
21        # parameter vector
22        # theta.
23        # Hint: While debugging, it can be useful to print out the
24        # values
25        # of the cost function (computeCost) and gradient
26        # here.
27        #
28
29        h = X @ theta
30        grad = (1/m) * (X.T @ (h - y))
31        theta = theta - alpha * grad
32
33        cost_history[i] = computeCostMulti(X, y, theta)
34        theta_history[:,i] = theta.reshape((n,))
35
36        # =====
37
38    return theta, cost_history, theta_history
```



3.2.2 Selecting learning rates



pouquoi ????

3.2.3 Predict the price for a new house

```

1 import numpy as np
2
3 # Estimate the price of a 1650 sq-ft, 3 br house
4 def predict_multi(house_features, mu, sigma, theta):
5

```

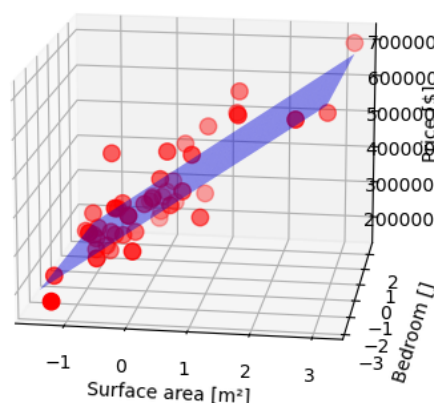
```

6      # ===== YOUR CODE HERE
7      # You should use the learned model with its
8      # characteristics (mu, sigma, theta)
9      # to predict the price of a house based on its features (
10     surface and bedrooms)
11
12     house_features_norm = (house_features - mu) / sigma
13     house_features_norm = np.concatenate((np.ones((1, 1)),
14     house_features_norm), axis=1)
15     price = house_features_norm.dot(theta)
16     print(f'shapes {np.shape(theta)} {np.shape(
17     house_features_norm)}')
18     return price

```

Predicted price of a 1650 sq-ft, 3 br house (using gradient descent): 293081.46

Plot to visualize the plan created by the function theta



3.3 Optional exercise: Normal Equations

```

1  import numpy as np
2
3
4  def normalEqn(X,y):
5      """ Computes the closed-form solution to linear regression
6          normalEqn(X,y) computes the closed-form solution to
7          linear
8          regression using the normal equations.
9      """
10

```

```
11 # ===== YOUR CODE HERE
12 # =====
13 # Instructions: Complete the code to compute the closed
14 # form solution
15 # to linear regression and put the result in
16 # theta.
17
18 t = np.linalg.pinv(X.T @ X) @ X.T @ y
19
20 # =====
21 return t
```

Solving with normal equations...

Theta computed from the normal equations: $\begin{bmatrix} 89597.90954355 & 139.21067402 \\ -8738.01911255 \end{bmatrix}$

Predicted price of a 1650 sq-ft, 3 br house (using normal equations): 293081.464335